

EPCNN: Efficient and Practical Privacy-Preserving Convolutional Neural Network Inference

Gang He , *Graduate Student Member, IEEE*, Yanli Ren , Jun Zhao , *Member, IEEE*, Guorui Feng ,
and Xinpeng Zhang 

Abstract—Convolutional Neural Networks (CNNs), as a powerful tool for efficient inference, have rapidly developed into a Machine Learning as a Service paradigm facilitated by cloud computing. Nevertheless, this service model raises privacy concerns, particularly in scenarios where relying on two non-colluding servers is unfeasible. To address this issue, we present EPCNN, an Efficient and Practical CNN inference scheme, which concurrently ensures data privacy, model privacy, and inference results with only a single server. EPCNN leverages Paillier homomorphic encryption for secure convolution operations on encrypted data and involves minimal client-server interactions. The client only participates in evaluating non-linear activation functions by judging the signs of blinded convolution results to streamline the interactions and enhance the system's practicality. Our security analysis validates the reliability of the proposed scheme, while experimental results demonstrate high inference accuracy comparable to plaintext-based methods. Compared to the state-of-the-art work, EPCNN attains huge improvements in both runtime and communication overhead.

Index Terms—CNN inference, cloud computing, privacy-preserving, homomorphic encryption, single server.

I. INTRODUCTION

COMPARED to traditional algorithms and other neural networks, convolutional neural networks (CNNs) have showcased remarkable proficiency in processing two-dimensional local information within images and extracting valuable image features. This prowess has resulted in the widespread adoption of CNNs across various applications, including image classification, target detection, and image generation [1].

As research progresses, the structure of network models is gradually deepened. The hardware limitations and computational demands make it unaffordable for individuals or micro-enterprises. To deal with this situation, the notion of Machine Learning as a Service (MLaaS) [2] has emerged and received widespread attention. The combination of MLaaS and cloud

computing enables the cloud platforms to offer various services to demanders through well-trained models.

For example, in a typical E-Medical scenario, the hospital can deploy a powerful inference model to the cloud, trained on the medical histories of past patients. This model then serves as a rapid diagnostic tool for subsequent patients, who upload their personal health summaries to the cloud. The hospital employs a specialized CNN model for pathological diagnosis to provide swift and accurate inference services. The diagnosis outcomes are subsequently relayed back to the patients [3].

While this approach seems promising, it also raises serious privacy concerns. First, the raw data provided by patients during queries often encompasses sensitive information, such as delicate medical conditions and personal details. Consequently, individuals are hesitant to disclose their private data publicly while seeking inference services. Moreover, a well-trained medical CNN model stands as a valuable asset for hospitals, necessitating safeguards against malicious entities aiming to misappropriate the model for unauthorized purposes. Thus, safeguarding the privacy of user data, models, and final inference results is a pivotal aspect of cloud-based CNN inference services [4].

Current approaches to privacy-preserving CNN inference primarily rely on homomorphic encryption and secret sharing techniques [5], [6], [7], [8], [9], [10], [11]. Specifically, homomorphic encryption realizes the same inference effect on ciphertext as the corresponding plaintext through the homomorphic properties [12], while secret sharing splits sensitive data into random shares engaging multiple participants in the inference process [13], [14], [15]. However, these methods face significant limitations:

- 1) Homomorphic encryption supports only linear arithmetic operations, necessitating supplementary techniques, such as garbled circuits [16] or polynomial approximations, to handle non-linear activation functions. While garbled circuits effectively handle non-linearities, they introduce high computational complexity and latency. Polynomial approximations simplify computation but reduce inference accuracy. Higher-order polynomials can mitigate this loss but at the cost of increased computational overhead.
- 2) Secret sharing requires multiple participants to reconstruct the data, and its security depends on preventing collusion among these participants, which may not always be feasible. Furthermore, when malicious attackers intercept the shared data, they can easily recover the original data and compromise privacy.

Received 20 May 2024; revised 20 January 2025; accepted 23 January 2025. Date of publication 28 January 2025; date of current version 25 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62476164 and in part by the Natural Science Foundation of Shanghai under Grant 22ZR1481000. Recommended for acceptance by Dr. Dusit Niyato. (Corresponding author: Yanli Ren.)

Gang He, Yanli Ren, Guorui Feng, and Xinpeng Zhang are with the School of Communication and Information Engineering, Shanghai University, Shanghai 200444, China (e-mail: hegang@shu.edu.cn; renyanli@shu.edu.cn; grfeng@shu.edu.cn; xzhang@shu.edu.cn).

Jun Zhao is with the School of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: junzhao@ntu.edu.sg).

Digital Object Identifier 10.1109/TNSE.2025.3534834

TABLE I
COMPARISON OF PRIVACY-PRESERVING SCHEMES FOR NEURAL NETWORK INFERENCE

Scheme	System Model	Cryptographic Primitive	Data Privacy	Model Privacy	Output Privacy	No Approximation	Computational Efficiency	Communication Efficiency
CryptoNets [12]	Client-Server	HE	● ²	○	●	○	Low ³	Low
Gazelle [17]	Client-Server	HE,GC,SS ¹	●	○	●	●	High	High
MUSE [18]	Client-Server	HE,GC,SS	●	●	●	●	Medium	Low
LPFHE [19]	Client-Server	HE	●	●	●	○	Medium	Medium
Sonic [20]	Two Servers	SS	○	○	○	●	Medium	Medium
Pio [21]	Two Servers	HE,GC,SS	○	●	○	●	High	High
Zhao et al. [22]	Two Servers	SS	○	○	○	●	High	High
PCCS [23]	Two Servers	HE	●	●	●	○	Low	Low
EPCNN (our work)	Client-Server	HE	●	●	●	●	Medium	High

¹ HE: Homomorphic Encryption; GC: Garbled Circuits; SS: Secret Sharing.

² ●: conditions are met; ○: conditions are not met; ○: conditions are partially met.

³ The efficiency levels in the table are relative to the EPCNN scheme we proposed.

Hybrid schemes also exist that combine homomorphic encryption and secret sharing, striking a balance between security and performance. Nevertheless, homomorphic encryption schemes often require an additional participant to decrypt intermediate results, while secret sharing necessitates the involvement of at least two participants. Consequently, privacy-preserving CNN inference methods can be categorized into dual-server and client-server modes. Dual-server schemes assume non-collusion between servers—a condition difficult to ensure in real-world settings. Scenarios involving more than two servers are rarely practical and are excluded from this discussion. Meanwhile, client-server schemes, though avoiding non-collusion assumptions, impose significant computational and communication burdens on the client.

Given these considerations, there is an urgent requirement for an efficient and practical approach to secure CNN inference. To tackle these limitations, we introduce EPCNN, a client-server mode CNN inference framework designed to minimize interaction overhead and enhance security. The security of EPCNN is ensured by the Paillier homomorphic encryption algorithm. Furthermore, EPCNN addresses the dual challenges of accuracy loss and computational overhead associated with non-linear activation functions like ReLU by engaging the client in evaluating the sign of blinded convolution outcomes. This novel approach eliminates the need for approximation-based activation functions and significantly reduces computational costs, ensuring accurate inference while preserving data and model privacy. Therefore, EPCNN achieves a robust balance between security and efficiency, offering enhanced privacy protection for sensitive data across all parties while minimizing client-side computational burdens through limited interactions.

Related work: We offer a comprehensive review of previous research regarding privacy-preserving neural network inference. For clarity, we perform a comparative analysis between our proposed scheme and representative existing methods, summarizing the key distinctions in Table I.

CryptoNets [12], proposed by Dowlin et al. in 2016, was the pioneering solution in realizing privacy-preserving deep learning inference through the application of cryptographic techniques. CryptoNets utilizes leveled homomorphic encryption to protect data and delivers results in encrypted form. However, it cannot guarantee the privacy of model parameters.

Moreover, CryptoNets uses a squared function to approximate the non-linear activation, which leads to inefficiency and reduces prediction accuracy.

In 2018, Juvekar et al. introduced Gazelle [17], an efficient framework designed for low-latency secure CNN inference. Gazelle utilizes packed additively homomorphic encryption [24] for the linear layers of CNN and employs garbled circuits for the non-linear layers. To ensure secure communication, their scheme employs secret sharing for data exchange between the user and the server. Subsequently, Mishra et al. introduced Delphi [25] as an improvement to Gazelle. Delphi shifts the cryptographic operations to the pre-processing phase, reducing the computational cost of the linear layer during the online phase. It also replaces ReLU functions with polynomial approximations to reduce costs. While these schemes claim to achieve perfect privacy for various data, in 2021, Lehmkuhl et al. proposed MUSE [18], which designed a model extraction attack specifically targeting semi-honest secure inference protocols based on additive secret sharing. This innovation raises significant challenges to the model privacy concerns addressed in previous similar work. To resist this model extraction attack, MUSE introduces message authentication codes, authenticated secret shares, and zero-knowledge proofs, which greatly impacts the efficiency, especially the additional homomorphic operations and zero-knowledge proofs that cause substantial communication costs. Recently, Wan et al. introduced LPFHE [19], a scheme based on RNS-CKKS encryption that employs low-complexity polynomials to achieve precise approximations of the ReLU function.

Another category of privacy-preserving CNN inference schemes is the dual-server-based approach, where either user data or model parameters are divided into two shares through secret sharing and subsequently uploaded to the servers [20], [21], [26], [27], [28]. Although these schemes eliminate the need for user involvement during the inference phase, as previously highlighted, they are vulnerable to interception and recovery by malicious attackers. Among these solutions, the approach introduced by Huang et al. [26], primarily focuses on partitioning data for sharing but retaining public model parameters. In contrast, Sonic [20] and PCNN_{CEC} [29] further split the model. Zhao et al. [22] design some lightweight secure building blocks on two edge cloud servers. These schemes, which rely on

secret sharing techniques, all demonstrate high efficiency. However, it's essential to recognize that deploying multiple servers remains challenging in real-world applications due to security assumptions and implementation constraints. Therefore, the latest proposed PCCS [23] employs DT-PKC encryption across two servers to ensure privacy.

Our contributions: We present EPCNN, a CNN inference scheme designed for efficiency and practicality while ensuring privacy preservation, which offers the following four key contributions:

- 1) *Enhanced privacy preservation:* EPCNN ensures privacy throughout the CNN inference process. In EPCNN, the server processes relevant computations on encrypted data, and homomorphic encryption guarantees the security of query data and inference outcomes. Moreover, EPCNN restricts the client's access to solely the blinded convolution results, preventing any attempts to deduce model information from intermediate computations. This design effectively preserves the security of client query input, inference outcomes, and the server's model parameters, shielding each party's data from mutual exposure.
- 2) *Streamlined interaction in single-server environments:* EPCNN significantly reduces interaction complexity in single-server configurations, only depending on the quantity of non-linear layers. In contrast to dual-server setups prone to unreliability, our client-server mode optimizes efficiency by minimizing interactions between the client and server. Linear computations, such as convolutional and fully-connected layers, proceed without interactions, while each non-linear activation function requires just one round of interaction. This streamlined approach enhances the practicality and reliability of our proposed scheme.
- 3) *Precise handling of non-linear functions:* EPCNN effectively addresses non-linear functions while maintaining high accuracy and privacy. Previous approaches employed polynomial approximations or garbled circuits to handle activation functions, which frequently resulted in compromises in either accuracy or efficiency. In contrast, EPCNN elegantly tackles the ReLU function by allowing the client to judge positive and negative signs of blinded convolution results. Experimental findings demonstrate that EPCNN achieves inference accuracy comparable to that of plaintext data.
- 4) *Experimental efficiency validation:* The experimental results validate the efficiency improvements realized by EPCNN. Compared to the state-of-the-art scheme PCCS, EPCNN demonstrates a 14.20-fold reduction in runtime and a significant improvement in communication efficiency.

Organization: The paper is organized as follows: Section II introduces fundamental concepts related to our proposed scheme. A detailed description of our privacy-preserving CNN inference scheme, EPCNN, is provided in Section III. Subsequently, we conduct a comprehensive analysis of EPCNN in Section IV, followed by experimental evaluations and comparative results in Section V. Finally, we conclude this paper in Section VI.

TABLE II
KEY NOTATIONS USED IN THIS PAPER

Notation	Description
gcd	Greatest common divisor
lcm	Least common multiple
pk	Public key of Paillier Encryption
sk	Private key of Paillier Encryption
$E_{pk}()$	Encrypted ciphertext under the public key pk
$D_{sk}()$	Decrypted plaintext under the private key sk
X	Query image matrix
C	Convolution kernel matrix
W	Fully-connected layer weight matrix
R	Random positive integer matrix
A	Output matrix from the activation layer
P	Output matrix from the pooling layer

II. PRELIMINARIES

This section offers a concise introduction to CNNs and Paillier homomorphic encryption algorithms employed in constructing our proposed scheme. Furthermore, we outline the system model, threat model, and design objectives, which serve as a foundation for our subsequent discussion of EPCNN. For clarity, Table II summarizes the key notations used in the paper.

A. Convolutional Neural Networks

In neural network architectures, layers of neurons are interconnected for data transmission, reception, and processing. CNNs, through parameter sharing mechanisms, allow the number of network parameters to be greatly reduced while ensuring better model performance. Consequently, CNNs have gained widespread applications across diverse areas. A common CNN architecture comprises the following four components:

- *Convolutional layer:* This layer performs a convolution operation on the input data using a convolution kernel to extract local features. Specifically, for the (j, k) -th neuron in the layer, with a convolution kernel of size $m \times m$, the output is computed as $y_{j,k} = \sum_{l=0}^{m-1} \sum_{n=0}^{m-1} c_{l,n} x_{j+l,k+n} + b$, where $c_{l,n}$ represents the parameter of the convolution kernel, $x_{j+l,k+n}$ denotes the input from the previous layer, and b is the shared bias term.
- *Activation layer:* This layer is responsible for applying non-linear transformations to the output from the preceding layer, enabling the network to address complex problems that linear models alone cannot effectively handle. The activation function used in this paper is ReLU, namely $f(x) = \max(0, x)$.
- *Pooling layer:* This layer performs downsampling on a specified range of neurons from the previous layer for the purpose of downscaling and removing redundant information. In this paper, we use average pooling, a method that computes the average values within the specified region.
- *Fully-connected layer:* Neurons in this layer establish connections with all neurons in the preceding layer, amalgamating features and acting as classifiers. Let $\omega_{j,k}$ represent the weight parameter connecting the k -th neuron in the preceding layer to the j -th neuron in the current layer, and b_j denotes the bias associated with the j -th neuron

in this layer. Consequently, the output of the j -th neuron is computed as $y_j = \sum_k \omega_{j,k} x_k + b_j$.

B. Paillier Homomorphic Encryption

The Paillier homomorphic encryption algorithm [30], proposed by Paillier in 1999, is a public key cryptosystem that supports additive homomorphism. Among many partially homomorphic encryption (PHE) schemes, the Paillier homomorphic encryption algorithm stands out in practical applications due to its high efficiency and complete security proof. It ranks among the most frequently employed PHE schemes in scenarios involving privacy-preserving computations.

The Paillier homomorphic encryption encompasses three core algorithms: key generation, encryption, and decryption, which are elaborated upon as follows:

- **KeyGen()**: Randomly select two large prime numbers p and q , such that they both have the same bit length and satisfy $\gcd(pq, (p-1)(q-1)) = 1$. Compute $n = pq$ and $\lambda = \text{lcm}(p-1, q-1)$ where lcm represents the least common multiple. Randomly choose an integer $g \leftarrow Z_{n^2}^*$. Define the function $L(x)$ as $L(x) = \frac{x-1}{n}$ and compute $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$.

$$pk = (n, g), sk = (\lambda, \mu) \quad (1)$$

- **Encrypt()**: Input a plaintext message m satisfying $0 \leq m < n$. Select a random number r to satisfy $0 \leq r < n$ and $r \in Z_n^*$. Compute the ciphertext

$$c = g^{m r^n} \bmod n^2. \quad (2)$$

- **Decrypt()**: Input the ciphertext c satisfying $c \in Z_{n^2}^*$ and compute the corresponding plaintext message

$$m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n. \quad (3)$$

The Paillier homomorphic encryption algorithm supports homomorphic addition and homomorphic scalar multiplication:

- **Add()**: Given the plaintexts m_1 and m_2 ,

$$E_{pk}(m_1) \otimes E_{pk}(m_2) = E_{pk}(m_1 + m_2) \bmod n^2. \quad (4)$$

- **ScalaMul()**: Given the plaintext m and scalar a ,

$$E_{pk}(m)^a = E_{pk}(a \cdot m) \bmod n^2. \quad (5)$$

C. System Model

Fig. 1 provides an overview of our proposed efficient and practical CNN inference scheme EPCNN. Our scheme mainly consists of two entities: the client, denoted as $\mathcal{C}$, and the server, denoted as $\mathcal{S}$. The client $\mathcal{C}$ serves as a query user and initiates the inference service. The server $\mathcal{S}$ possesses a proficiently trained CNN model and powerful computational resources, responsible for executing the primary operations of the inference service.

Prior to the inference stage, the client $\mathcal{C}$ encrypts its original data to be inferred and subsequently transmits the resulting ciphertext to the server $\mathcal{S}$, as shown in Fig. 1. Subsequently, the server $\mathcal{S}$ undertakes CNN inference with the cooperation of the client $\mathcal{C}$. Specifically, the server $\mathcal{S}$ first conducts convolutional operations on the received ciphertexts based on the homomorphism properties of Paillier. It then obtains the outcomes after

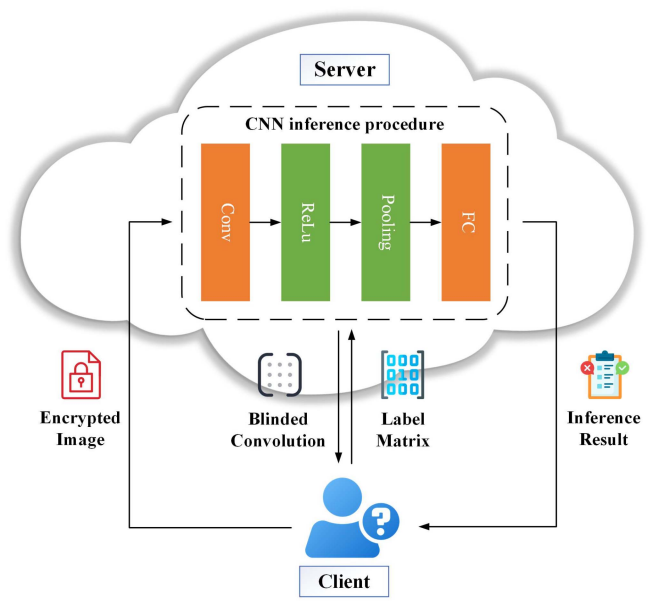


Fig. 1. System model.

the ReLU activation function with the assistance of the client $\mathcal{C}$ and proceeds to execute the subsequent pooling operations. Finally, the server $\mathcal{S}$ returns the ciphertext inference results after the fully-connected layer to the client $\mathcal{C}$. The client $\mathcal{C}$ can obtain the final inference results after decryption.

D. Threat Model and Design Objectives

Threat model: Our scheme encompasses two distinct categories of potential adversaries: the honest but curious client and the honest but curious server.

- **Honest but curious client:** In EPCNN, the client $\mathcal{C}$ will honestly encrypt its query data without tampering, but will be curious about the intermediate results received from the server $\mathcal{S}$ during the inference phase and try to recover model parameters from them.
- **Honest but curious server:** Similarly, the server $\mathcal{S}$ will faithfully perform the entire inference process without tampering, but will be curious about the data obtained from the client $\mathcal{C}$ and make attempts to recover the client's data and final inference results.

Design objectives: Combined with the above models, our scheme should meet the following four design objectives:

- **Privacy:** The original data and inference results of the client $\mathcal{C}$ should not be exposed to the server $\mathcal{S}$. Simultaneously, the model parameters belonging to the server $\mathcal{S}$ should not be leaked to the client $\mathcal{C}$.
- **Practicality:** For practical deployment, the number of interactions during the scheme execution should be minimized to facilitate the user.
- **Accuracy:** Although the loss of inference accuracy caused by ciphertext operations is inevitable, it is crucial to strive for maintaining inference accuracy that is reasonably comparable to plaintext inference.

- *Efficiency*: In comparison to existing schemes, the performance of the proposed scheme should maintain high efficiency, characterized by lower computational costs and less communication overhead.

III. PROPOSED EPCNN SCHEME

In this section, we present our EPCNN scheme. The comprehensive framework comprises six stages, which we shall elaborate on subsequently.

A. Initialization

In this initial stage, given the security parameters κ , the client $\mathcal{C}$ initializes the system and distributes keys. Specifically, the client $\mathcal{C}$ executes the algorithm **KeyGen**() to generate public parameters. The public key pk is openly accessible, while the private key sk is securely held in private. Subsequently, the client $\mathcal{C}$ preprocesses its query data.

Assuming the client $\mathcal{C}$ holds an $n \times n$ query image $X = (x_{1,1}, x_{1,2}, \dots, x_{1,n}, x_{2,1}, \dots, x_{n,n})$, and the server $\mathcal{S}$ possesses an $m \times m$ convolution kernel $C = (c_{1,1}, c_{1,2}, \dots, c_{1,m}, c_{2,1}, \dots, c_{m,m})$ and a fully-connected layer weight matrix $W = (w_{1,1}, w_{1,2}, \dots, w_{1,k}, w_{2,1}, \dots, w_{l,k})$ of dimensions $l \times k$, where $n > m$. The inclusion of bias parameters in convolutional and fully-connected layers is omitted, as their effects can be offset through normalization operations without impacting the overall scheme. Additionally, the normalized query image and model parameters are all floating-point numbers, while Paillier solely supports operations on integer data, so floating-point data needs to be converted to integer data. Taking the normalized query image X as an example, the specific operation is to select a rounding factor a , and execute $x_{i,j} \leftarrow \lfloor a \times x_{i,j} \rfloor$ for all elements in X , where $0 \leq i, j \leq n$ and $\lfloor \cdot \rfloor$ denotes the rounding down operation. For the convenience of expression, subsequent floating-point data are treated as rounded values.

The client $\mathcal{C}$ executes the algorithm **Encrypt**() to encrypt the query image X via the public key pk . The outcome is a set of ciphertext data denoted as $E_{pk}(X) = (E_{pk}(x_{1,1}), E_{pk}(x_{1,2}), \dots, E_{pk}(x_{1,n}), E_{pk}(x_{2,1}), \dots, E_{pk}(x_{n,n}))$. Subsequently, both the ciphertext data $E_{pk}(X)$ and the rounding factor a are transmitted to the server $\mathcal{S}$.

Algorithm 1 outlines the specific steps involved in the initialization process.

B. Convolution Layer

Upon the server $\mathcal{S}$ receiving the ciphertext $E_{pk}(X)$, the convolution layer algorithm is executed.

The operations performed are similar to those on the plaintext. The server $\mathcal{S}$ first extracts sub-images of the same dimensions as the convolution kernel C from the ciphertext image $E_{pk}(X)$. Then, the server $\mathcal{S}$ performs homomorphic scalar multiplications on the elements of C with the corresponding elements from the ciphertext sub-image at each position and computes the sum. The resulting value is stored at the corresponding position of the

Algorithm 1: Initialization.

Input: Security parameter κ , query image $X \in \mathbb{R}^{n \times n}$ (held by client $\mathcal{C}$), convolution kernel $C \in \mathbb{R}^{m \times m}$, fully-connected layer weight matrix $W \in \mathbb{R}^{l \times k}$ (held by server $\mathcal{S}$).

Output: Encrypted query image $E_{pk}(X)$ and rounding factor a .

1 Client $\mathcal{C}$:

```

2   Generate public and private keys:
     $(pk, sk) \leftarrow \text{KeyGen}(\kappa)$ ;
3   Choose a rounding factor  $a$  and compute
4    $x_{i,j} \leftarrow \lfloor a \times x_{i,j} \rfloor \quad \forall x_{i,j} \in X$ ;
5   Encrypt query image using public key:
6    $E_{pk}(X) \leftarrow \text{Encrypt}(pk, X)$ ;
7   Transmit  $E_{pk}(X)$  and  $a$  to server  $\mathcal{S}$ ;

```

8 Server $\mathcal{S}$:

```

9   Receive encrypted query image  $E_{pk}(X)$  and
    rounding factor  $a$ ;
10  For each element  $c_{i,j} \in C$ :
11     $c_{i,j} \leftarrow \lfloor a \times c_{i,j} \rfloor$ ;
12  For each element  $w_{i,j} \in W$ :
13     $w_{i,j} \leftarrow \lfloor a \times w_{i,j} \rfloor$ ;
14  Hold  $C$  and  $W$  for subsequent computations;

```

output. This procedure is reiterated until the whole ciphertext image $E_{pk}(X)$ has been traversed.

Next, we will provide a detailed explanation of the ciphertext convolution algorithm. First, the server $\mathcal{S}$ performs homomorphic scalar multiplications based on C and a ciphertext sub-image of dimensions $m \times m$. The specific operations are carried out as follows:

$$\begin{aligned}
E_{pk}(x_{i,j})^{c_{1,1}} &= E_{pk}(x_{i,j} \cdot c_{1,1}); \\
E_{pk}(x_{i,j+1})^{c_{1,2}} &= E_{pk}(x_{i,j+1} \cdot c_{1,2}); \\
&\dots \\
E_{pk}(x_{i,j+m-1})^{c_{1,m}} &= E_{pk}(x_{i,j+m-1} \cdot c_{1,m}); \\
E_{pk}(x_{i+1,j})^{c_{2,1}} &= E_{pk}(x_{i+1,j} \cdot c_{2,1}); \\
&\dots \\
E_{pk}(x_{i+m-1,j+m-1})^{c_{m,m}} &= E_{pk}(x_{i+m-1,j+m-1} \cdot c_{m,m}). \tag{6}
\end{aligned}$$

Subsequently, the server $\mathcal{S}$ performs homomorphic additions based on the calculation results of the ciphertext homomorphic scalar multiplications:

$$\begin{aligned}
&E_{pk}((c * x)_{i,j}) \\
&= E_{pk}(x_{i,j} \cdot c_{1,1}) \otimes \dots \otimes E_{pk}(x_{i+m-1,j+m-1} \cdot c_{m,m}) \\
&= E_{pk}(x_{i,j} \cdot c_{1,1} + \dots + x_{i+m-1,j+m-1} \cdot c_{m,m}). \tag{7}
\end{aligned}$$

The resulting value $E_{pk}((c * x)_{i,j})$ is stored at the corresponding position in the convolution matrix $E_{pk}(C * X)$. The server $\mathcal{S}$ performs the ciphertext convolution algorithm on the sub-images one by one according to a certain stride and padding until

Algorithm 2: Convolution Layer.

Input: Encrypted query image $E_{pk}(X)$, convolution kernel C , stride s , padding p
Output: Ciphertext matrix $E_{pk}(C * X)$ after convolution

```

1 Server S:
2   for each sub-image in  $E_{pk}(X)$  with stride  $s$  and padding  $p$  do
3     for each element in the sub-image do
4       Perform homomorphic scalar multiplication:
5        $E_{pk}(x_{i,j})^{c_{1,1}} = E_{pk}(x_{i,j} \cdot c_{1,1})$ 
6        $E_{pk}(x_{i,j+1})^{c_{1,2}} = E_{pk}(x_{i,j+1} \cdot c_{1,2})$ 
7       ...
8        $E_{pk}(x_{i+m-1,j+m-1})^{c_{m,m}} =$ 
9        $E_{pk}(x_{i+m-1,j+m-1} \cdot c_{m,m})$ 
9     Perform homomorphic addition:
10     $E_{pk}((c * x)_{i,j}) = E_{pk}(x_{i,j} \cdot c_{1,1}) \otimes \dots \otimes$ 
11     $E_{pk}(x_{i+m-1,j+m-1} \cdot c_{m,m})$ 
11    Store result in  $E_{pk}((c * x)_{i,j})$  at corresponding position
12  return  $E_{pk}(C * X)$ 

```

the entire image is convoluted. Finally, the complete convolution results is a ciphertext matrix of dimensions $((n - m + 2p)/s + 1) \times ((n - m + 2p)/s + 1)$, where p denotes the padding value and s denotes the stride value.

The detailed procedure for performing convolution operations is outlined in Algorithm 2.

C. Activation Layer

The activation function used in our scheme is ReLU, denoted as $f(x) = \max(0, x)$. It outputs the input itself for positive values and 0 for non-positive cases. Therefore, determining the output results simply involves judging the positivity and negativity. Since the server S alone faces difficulty in distinguishing the positivity and negativity aspects of the convolutional layer's output on ciphertext, the activation layer algorithm needs the assistance of the client C .

Specifically, after obtaining the ciphertext convolution matrix $E_{pk}(C * X)$ from the convolutional layer, the server S first generates a random positive integer matrix R with the same dimensions as $E_{pk}(C * X)$. Subsequently, homomorphic scalar multiplications are executed on the corresponding positions of these two matrices, expressed as:

$$E_{pk}(c * x)_{i,j}^{r_{i,j}} = E_{pk}(r_{i,j} \cdot (c * x)_{i,j}). \quad (8)$$

where $1 \leq i, j \leq ((n - m + 2p)/s + 1)$. After homomorphic scalar multiplications at all positions are performed, the result is expressed as $E_{pk}(R \odot (C * X))$, where $\odot$ represents the Hadamard product. This operation is carried out to ensure that subsequent decryption by the client C does not disclose any information related to the model parameters. Subsequently, the server S sends the blinded ciphertext convolution matrix $E_{pk}(R \odot (C * X))$ to the client C , who uses its private key

Algorithm 3: Activation Layer (ReLU).

Input: Ciphertext convolution matrix $E_{pk}(C * X)$
Output: Ciphertext matrix $E_{pk}(A)$ after ReLU activation

```

1 Server S:
2   Generate random positive integer matrix  $R$  with the same dimensions as  $E_{pk}(C * X)$ ;
3   Perform homomorphic scalar multiplication:
4   for each element  $(c * x)_{i,j}$  in  $E_{pk}(C * X)$  do
5     Compute:
6      $E_{pk}(c * x)_{i,j}^{r_{i,j}} = E_{pk}(r_{i,j} \cdot (c * x)_{i,j})$ 
7   Send blinded ciphertext matrix  $E_{pk}(R \odot (C * X))$  to client  $C$ ;
8 Client C:
9   Decrypt the blinded ciphertext matrix:
10   $R \odot (C * X) = \text{Decrypt}(sk, E_{pk}(R \odot (C * X)))$ ;
11  for each element  $(r_{i,j} \cdot (c * x)_{i,j})$  in  $R \odot (C * X)$  do
12    if  $r_{i,j} \cdot (c * x)_{i,j} > 0$  then
13       $L_{i,j} \leftarrow 1$ ;
14    else
15       $L_{i,j} \leftarrow 0$ ;
16  Send the label matrix  $L$  to server  $S$ ;
17 Server S:
18  Receive label matrix  $L$  from client  $C$ ;
19  Perform Hadamard product:
20   $E_{pk}(A) = E_{pk}(C * X) \odot L$ ;
21  return  $E_{pk}(A)$ 

```

sk to execute the algorithm $\text{Decrypt}()$, resulting in $R \odot (C * X)$. Since R consists of solely positive values, the sign of the elements in $R \odot (C * X)$ is the same as that of the elements in $C * X$. Therefore, the output results after the ReLU activation function can be determined by judging the sign of the elements in $R \odot (C * X)$. We represent the elements where $r_{i,j} \cdot (c * x)_{i,j} > 0$ as 1 and the elements at other positions as 0, resulting in the label matrix L :

$$L = \begin{cases} 1, & r_{i,j} \cdot (c * x)_{i,j} > 0 \\ 0, & \text{others} \end{cases} \quad (9)$$

Finally, the client C sends the label matrix L back to the server S . Subsequently, the server S executes a Hadamard product operation between the label matrix L and the ciphertext convolution matrix $E_{pk}(C * X)$, represented as:

$$E_{pk}(A) = E_{pk}(C * X) \odot L. \quad (10)$$

In $E_{pk}(A)$, the ciphertext elements corresponding to plaintext values greater than 0 remain unchanged, while other positions are set to 0.

The steps for performing element-wise non-linear transformations on the ciphertext result from the convolution operation are detailed in Algorithm 3.

D. Pooling Layer

In our scheme, the pooling layer employs the average pooling algorithm. After obtaining $E_{pk}(\mathbf{A})$ from the activation layer, the server $\mathcal{S}$ proceeds to perform ciphertext average pooling on $E_{pk}(\mathbf{A})$.

Assuming a pooling window of dimensions $q \times q$, the server $\mathcal{S}$ selects ciphertext sub-images of dimensions $q \times q$ from $E_{pk}(\mathbf{A})$. It subsequently performs homomorphic additions and finally performs homomorphic scalar multiplications with the scalar q^{-2} . According to the homomorphic property of Paillier encryption, this process is equivalent to outputting the average value of plaintext within the corresponding region.

Specifically, for any given $q \times q$ region within $E_{pk}(\mathbf{A})$, the server $\mathcal{S}$ first performs homomorphic additions, namely

$$\begin{aligned} & E_{pk}(a_{i,j} + \dots + a_{i,j+q-1} + \dots + a_{i+q-1,j+q-1}) \\ &= E_{pk}(a_{i,j}) \otimes \dots \otimes E_{pk}(a_{i,j+q-1}) \otimes \dots \otimes E_{pk}(a_{i+q-1,j+q-1}). \end{aligned} \quad (11)$$

Next, the server $\mathcal{S}$ performs the homomorphic scalar multiplications to obtain the average value of the region expressed as

$$\begin{aligned} & E_{pk}(p_{i,j}) \\ &= E_{pk}(a_{i,j} + \dots + a_{i,j+q-1} + \dots + a_{i+q-1,j+q-1})^{q^{-2}} \\ &= E_{pk}(q^{-2} \cdot (a_{i,j} + \dots + a_{i,j+q-1} + \dots + a_{i+q-1,j+q-1})). \end{aligned} \quad (12)$$

Note that q^{-2} here is a floating-point value that also needs to be multiplied by the rounding factor a for rounding. The server $\mathcal{S}$ systematically traverses all regions within the ciphertext activation matrix $E_{pk}(\mathbf{A})$ based on a certain stride s and padding p , and performs the necessary computations for average pooling. Finally, the result matrix from the pooling layer is represented as $E_{pk}(\mathbf{P})$.

The detailed algorithm for the pooling layer is outlined in Algorithm 4.

E. Fully-Connected Layer

The algorithm for the fully-connected layer is similar to that of the convolutional layer, thus we will provide a concise overview without delving into specific details. In general, the server $\mathcal{S}$ first converts the received ciphertext pooling matrix $E_{pk}(\mathbf{P})$ into a vector form, and then performs homomorphic scalar multiplications and homomorphic additions with the column vectors of the fully-connected layer weight matrix $\mathbf{W}$, respectively. The final result $E_{pk}(\mathbf{WP})$ is sent to the client $\mathcal{C}$.

The detailed operations of the fully-connected layer are outlined in Algorithm 5.

F. Final Inference Decryption

Upon receiving the ciphertext inference result $E_{pk}(\mathbf{WP})$, the client $\mathcal{C}$ executes the algorithm **Decrypt()** with its private key sk for the decryption process:

$$D_{sk}(E_{pk}(\mathbf{WP})) = \mathbf{WP}. \quad (13)$$

Algorithm 4: Pooling Layer (Average Pooling).

Input: Ciphertext activation matrix $E_{pk}(\mathbf{A})$, pooling window size $q \times q$, stride s , padding p
Output: Ciphertext matrix $E_{pk}(\mathbf{P})$ after average pooling

```

1 Server  $\mathcal{S}$ :
2   for each  $q \times q$  region in  $E_{pk}(\mathbf{A})$  with stride  $s$  and padding  $p$  do
3     for each element  $(a_{i,j})$  in the  $q \times q$  region do
4       Perform homomorphic additions:
5        $E_{pk}(a_{i,j} + \dots + a_{i+q-1,j+q-1}) =$ 
6        $E_{pk}(a_{i,j}) \otimes \dots \otimes E_{pk}(a_{i+q-1,j+q-1})$ 
7       Perform homomorphic scalar multiplication by  $q^{-2}$  to obtain the average:
8        $E_{pk}(p_{i,j}) =$ 
9        $E_{pk}(q^{-2} \cdot (a_{i,j} + \dots + a_{i+q-1,j+q-1}))$ 
10    return  $E_{pk}(\mathbf{P})$ 

```

Algorithm 5: Fully-Connected Layer.

Input: Ciphertext pooling matrix $E_{pk}(\mathbf{P})$, weight matrix $\mathbf{W} \in \mathbb{R}^{l \times k}$
Output: Ciphertext matrix $E_{pk}(\mathbf{WP})$ after fully connected

```

1 Server  $\mathcal{S}$ :
2   Convert the ciphertext pooling matrix  $E_{pk}(\mathbf{P})$  into a column vector form,  $\mathbf{P}'$ .
3   for each row vector  $\mathbf{w}_i$  of the weight matrix  $\mathbf{W}$  (where  $i = 1 \dots l$ ) do
4     for each element  $p_j$  in the column vector  $\mathbf{P}'$  (where  $j = 1 \dots k$ ) do
5       Perform homomorphic scalar multiplication:
6        $E_{pk}(p_j)^{w_{i,j}} = E_{pk}(w_{i,j} \cdot p_j)$ 
7       Perform homomorphic addition for the scalar products:
8        $E_{pk}(z_i) = E_{pk}(\sum_j w_{i,j} \cdot p_j)$ 
9   return  $E_{pk}(\mathbf{WP})$ 

```

Finally, the client $\mathcal{C}$ is required to execute an inverse transformation on the previous rounding and magnification multiples to obtain the final real inference result $\mathbf{WP}$.

The decryption process is detailed in Algorithm 6.

IV. THEORETICAL ANALYSIS

This section provides a detailed theoretical analysis of our proposed EPCNN framework, encompassing its security properties, computational complexity, and communication overhead.

A. Security Analysis

The security of our proposed scheme relies on the semantic security of Paillier encryption, which safeguards against chosen-plaintext attacks (IND-CPA security). We establish the

Algorithm 6: Final Inference Decryption.

Input: Ciphertext inference result $E_{pk}(WP)$, private key sk , rounding factor a

Output: Final real inference result WP

```

1 Client C:
2   Decrypt the ciphertext inference result:
3    $WP \leftarrow D_{sk}(E_{pk}(WP))$ .
4   Perform inverse transformation to reverse the
   rounding and magnification operations:
5   for each element  $(wp)_i$  in the vector  $WP$  do
6     Reverse the rounding operation:
7      $(wp)_i \leftarrow \frac{(wp)_i}{a^2}$ 
8   Return the final real inference result  $WP$ 

```

security with a hybrid argument that leans on simulators [31]. This approach effectively illustrates that neither the server $\mathcal{S}$ nor the client $\mathcal{C}$ gain access to any query data, model parameters, or inference results from each other's views during the execution of EPCNN.

Theorem 1 (Security against a curious server): For a given security parameter κ , let the random variable $\mathbf{REAL}_S^{C,\kappa}$ represent the view of server $\mathcal{S}$ during the actual execution of our scheme. Then, there exists a probabilistic polynomial-time (PPT) simulator, denoted as $\mathbf{SIM}$, such that the output of $\mathbf{SIM}$ is computationally indistinguishable from $\mathbf{REAL}_S^{C,\kappa}$:

$$\mathbf{REAL}_S^{C,\kappa} \equiv \mathbf{SIM}_S^{C,\kappa}$$

where “ $\equiv$ ” denotes computational indistinguishability.

Proof: According to the definition of $\mathbf{REAL}_S^{C,\kappa}$, it includes all the information acquired within $\mathcal{S}$ throughout the execution of the scheme. Given the security parameter κ , we construct a PPT simulator $\mathbf{SIM}$ by introducing a series of modifications to the random variables in $\mathbf{REAL}_S^{C,\kappa}$. These modifications are conducted in a polynomially numerous manner, with the objective of rendering the output of $\mathbf{SIM}$ computationally indistinguishable from $\mathbf{REAL}_S^{C,\kappa}$. The detailed proof is presented below.

Hyb₁: We start by establishing an initial random variable with a distribution that is indistinguishable from $\mathbf{REAL}_S^{C,\kappa}$, representing the view within $\mathcal{S}$ throughout the actual execution of the scheme.

Hyb₂: In this hybrid, we modify the input of the convolutional layer executed by server $\mathcal{S}$ using $E_{pk}(X')$ and C' instead of $E_{pk}(X)$ and C . The server $\mathcal{S}$ holds the view $\mathbf{SIM}_S^{C,\kappa} = \{E_{pk}(C' * X')\}$. The properties of additive homomorphic and scalar multiplication homomorphic ensure that the distribution of any intermediate variables remains consistent with that of an equivalent number of intermediate variables. Since $E_{pk}(X')$ is encrypted, the server $\mathcal{S}$ does not disclose any privacy about the client $\mathcal{C}$ without knowledge of the input. The IND-CPA security of Paillier [30], along with the honest but curious server $\mathcal{S}$, ensures that this hybrid is indistinguishable from the previous one.

Hyb₃: In this hybrid, we simulate the server $\mathcal{S}$ by introducing a random matrix R' sampled uniformly at random to perturb the

convolution matrix $E_{pk}(C' * X')$, replacing the use of R . The parameters modified by adding uniformly random numbers also follow a uniform random distribution. As the distribution of the random matrix R' and the perturbed parameters are uniformly random, both this hybrid and the previous one are uniformly and randomly sampled from identical distribution. Moreover, the IND-CPA security of Paillier, along with the honest but curious server $\mathcal{S}$, ensures that this hybrid is indistinguishable from the previous one.

Hyb₄: In this hybrid, we replace the two matrices that the server $\mathcal{S}$ operates on in the activation layer, which is $E_{pk}(C * X)$ and L , with $E_{pk}(C' * X')$ and L' . As only the ciphertext contents change, the IND-CPA security of Paillier, along with the honest but curious server $\mathcal{S}$, ensures that this hybrid is indistinguishable from the previous one.

Hyb₅: Similar to **Hyb₄**, in this hybrid, we modify the input of the pooling layer executed by server $\mathcal{S}$ to be $E_{pk}(A')$ instead of $E_{pk}(A)$. As only the ciphertext contents change, the IND-CPA security of Paillier, along with the honest but curious server $\mathcal{S}$, ensures that this hybrid is indistinguishable from the previous one.

Hyb₆: Similar to **Hyb₂**, in this hybrid, we modify the input of the fully-connected layer executed by server $\mathcal{S}$ to be P' and W' instead of P and W . The Server $\mathcal{S}$ holds the view $\mathbf{SIM}_S^{C,\kappa} = \{E_{pk}(W'P')\}$. Due to the homomorphic properties and the encrypted matrix $E_{pk}(P')$, the server $\mathcal{S}$ cannot reveal the inference results about the client $\mathcal{C}$ without knowledge of the input. The IND-CPA security of Paillier, along with the honest but curious server $\mathcal{S}$, ensures that this hybrid is indistinguishable from the previous one.

Theorem 2 (Security Against a 100.urious 100.lient): For a given security parameter κ , let the random variable $\mathbf{REAL}_C^{S,\kappa}$ represent the view of client $\mathcal{C}$ during the actual execution of our scheme. There also exists a PPT simulator $\mathbf{SIM}$ whose output is computationally indistinguishable from $\mathbf{REAL}_C^{S,\kappa}$:

$$\mathbf{REAL}_C^{S,\kappa} \equiv \mathbf{SIM}_C^{S,\kappa}$$

Proof: Similar to Theorem 1, we create a PPT simulator $\mathbf{SIM}$ by performing a series of modifications to the random variables within $\mathbf{REAL}_C^{S,\kappa}$ to ensure that the output of $\mathbf{SIM}$ is computationally indistinguishable from $\mathbf{REAL}_C^{S,\kappa}$. The detailed proof is as follows.

Hyb₁: This process is similar to Theorem 1. We start by establishing an initial random variable with a distribution that is indistinguishable from $\mathbf{REAL}_C^{S,\kappa}$, representing the view within $\mathcal{C}$ throughout the actual execution of the scheme.

Hyb₂: In this hybrid, we modify the simulated client $\mathcal{C}$'s behavior to encrypt a randomly selected query image X' with the public key pk of Paillier, rather than the original query image X . As this change solely impacts the ciphertext contents, the IND-CPA security of Paillier, along with the honest but curious client $\mathcal{C}$, ensures that this hybrid is indistinguishable from the previous one.

Hyb₃: In this hybrid, we simulate the client $\mathcal{C}$ to generate the label matrix L' by determining the positive and negative via $R' \odot (C' * X')$ instead of $R \odot (C * X)$. Despite the client

$\mathcal{C}$ possessing the private key sk for decrypting the mentioned ciphertexts, the multiplication of the convolution results by random numbers remains in a uniformly random distribution. This uniform randomness is maintained because the intermediate parameters are masked by uniformly random numbers. Consequently, the client $\mathcal{C}$ cannot infer any model parameter information about the server $\mathcal{S}$ from the view $\text{SIM}_{\mathcal{C}}^{\mathcal{S}, \kappa} = \{\mathbf{R}' \odot (\mathbf{C}' * \mathbf{X}')\}$. Therefore, this hybrid is indistinguishable from the previous one.

The above argument demonstrates that a simulator SIM can be sampled from the previously stated distribution, resulting in its output being computationally indistinguishable from that of REAL . According to Hyb_2 , Hyb_6 of Theorem 1 and Hyb_3 of Theorem 2, the curious server $\mathcal{S}$ is unaware of the query data and inference results of the client $\mathcal{C}$, while the curious client $\mathcal{C}$ knows nothing about the model parameters of the server $\mathcal{S}$.

B. Computational Analysis

The computational complexity of the EPCNN framework is primarily determined by the operations related to the encryption algorithm. We define T_{Enc} , T_{Dec} , T_{SMul} , and T_{Add} as the time required for encryption, decryption, homomorphic scalar multiplication, and homomorphic addition, respectively. Let n denote the dimension of the query image, m the dimension of the convolution kernel, c_{in} and c_{out} the number of input and output channels in the convolution layer, q the pooling window size, p the padding size, and s the stride length. Additionally, let κ represent the security parameter, corresponding to the bit length of the encryption key.

- **Initialization:** The computational cost of this stage arises from the execution of $\text{Encrypt}()$. For c_{in} input channels, the total time required to encrypt each pixel of an $n \times n$ query image is: $c_{in}n^2T_{Enc}$.
- **Convolution Layer:** In the convolutional layer, the server $\mathcal{S}$ performs homomorphic scalar multiplications and additions for each sliding window. For a single kernel of size $m \times m$, m^2 homomorphic scalar multiplications and $m^2 - 1$ homomorphic additions are required. Considering $c_{in} \times c_{out}$ convolution kernel channels and an output feature map of dimension $u \times u$, the overall time of this layer is: $c_{in}c_{out}u^2[m^2T_{SMul} + (m^2 - 1)T_{Add}]$.
- **Activation Layer:** In the activation layer, the server $\mathcal{S}$ performs two Hadamard product operations, while the client $\mathcal{C}$ mainly performs decryption. The computational cost associated with generating the random matrix $\mathbf{R}$, constructing the label matrix $\mathbf{L}$, and judging the sign of the elements in $\mathbf{R} \odot (\mathbf{C} * \mathbf{X})$ is negligible. Consequently, for an output feature map of dimension $u \times u$ across c_{in} input channels, the total time required for the activation layer is: $c_{in}u^2(2T_{SMul} + T_{Dec})$.
- **Pooling Layer:** Average pooling requires $q^2 - 1$ homomorphic additions followed by one homomorphic scalar multiplication for each pooling window. For c_{in} input channels and an output feature map of dimension $u \times u$, the cost is: $c_{in}u^2[T_{SMul} + (q^2 - 1)T_{Add}]$.

- **Fully-Connected Layer:** The fully-connected layer performs homomorphic scalar multiplications and additions for each pair of weight and input vector elements. For a weight matrix of size $l \times k$ and an input vector of size l , the computational cost is: $lkT_{SMul} + (l - 1)kT_{Add}$.
- **Final Inference Decryption:** The client $\mathcal{C}$ decrypts each element of the result vector. For an output of size k , the total decryption time is: kT_{Dec} .

The most time-consuming operations across all stages involve modular exponentiations, with a computational complexity of $O(\kappa)$. By employing the fast modular exponentiation algorithm, the complexity is reduced to $O(\log \kappa)$. Consequently, the overall computational complexity of EPCNN is $O(\log \kappa)$.

C. Communication Analysis

The communication overhead in the EPCNN framework primarily arises from encrypted data transmission during the initialization and final inference decryption stages, as well as from client-server interactions in the activation layer. Given a security parameter κ , the size of each transmitted ciphertext is 2κ bits.

- **Initialization:** The client $\mathcal{C}$ sends the encrypted query image $E_{pk}(\mathbf{X})$ to the server $\mathcal{S}$. For c_{in} input channels and an image of size $n \times n$, the communication cost is: $2c_{in}n^2\kappa$.
- **Activation Layer:** The server $\mathcal{S}$ sends the blinded ciphertext matrix $E_{pk}(\mathbf{R} \odot (\mathbf{C} * \mathbf{X}))$ to the client $\mathcal{C}$, who then returns the label matrix $\mathbf{L}$. For c_{in} input channels, and an output feature map of size $u \times u$, the communication cost is: $4c_{in}u^2\kappa$.
- **Final Inference Decryption:** The server $\mathcal{S}$ transmits the encrypted inference result $E_{pk}(\mathbf{WP})$ to the client $\mathcal{C}$. For an output size k , the communication cost is: $2k\kappa$.

The theoretical computational and communication complexities of EPCNN across different stages are summarized in Table III.

V. EXPERIMENTAL RESULTS

In this section, we provide an in-depth evaluation of EPCNN through a series of experiments and comparisons. We begin by evaluating the accuracy of the model and examining its computational costs and communication overhead. Furthermore, to verify its performance, we select representative methods from recent privacy-preserving CNN inference schemes and conduct a comparative analysis with our EPCNN.

A. Experiment Setup

In preparation for the experiments, we configure the experimental environment, select the datasets, and establish the model network structures.

The simulation experiments are performed on a computer equipped with a 3.60 GHz 12th Gen Intel(R) Core(TM) i7-12700 K processor, 16 GB of RAM, and the Windows 11 operating system. The experiments are implemented by using Python as the programming language. For the Paillier homomorphic encryption algorithm, we utilize the PHE library to implement

TABLE III
THEORETICAL COMPUTATIONAL AND COMMUNICATION COMPLEXITY ANALYSIS OF EPCNN

Stage	Computational Complexity	Communication Complexity
Initialization	$c_{in}n^2T_{Enc}$	$2c_{in}n^2\kappa$
Convolution Layer	$c_{in}c_{out}u^2[m^2T_{SMul} + (m^2 - 1)T_{Add}]$	—
Activation Layer	$c_{in}u^2(2T_{SMul} + T_{Dec})$	$4c_{in}u^2\kappa$
Pooling Layer	$c_{in}u^2[T_{SMul} + (q^2 - 1)T_{Add}]$	—
Fully-Connected Layer	$lkT_{SMul} + (l - 1)kT_{Add}$	—
Final Inference Decryption	kT_{Dec}	$2k\kappa$

TABLE IV
NETWORK I: CNN ARCHITECTURE FOR MNIST DATASET

Layer	Parameters	Output size
Convolution Layer	$5 \times 5 \times 16, s = 1, p = 2$	$28 \times 28 \times 16$
Activation Layer	—	$28 \times 28 \times 16$
Pooling Layer	—	$14 \times 14 \times 16$
Convolution Layer	$5 \times 5 \times 32, s = 1, p = 2$	$14 \times 14 \times 32$
Activation Layer	—	$14 \times 14 \times 32$
Pooling Layer	—	$7 \times 7 \times 32$
Fully-Connected Layer	1568×10	1×10

TABLE V
NETWORK II: CNN ARCHITECTURE FOR CIFAR-10 DATASET

Layer	Parameters	Output size
Convolution Layer	$3 \times 3 \times 64, s = 1, p = 1$	$32 \times 32 \times 64$
Activation Layer	—	$32 \times 32 \times 64$
Pooling Layer	—	$16 \times 16 \times 64$
Convolution Layer	$3 \times 3 \times 128, s = 1, p = 1$	$16 \times 16 \times 128$
Activation Layer	—	$16 \times 16 \times 128$
Pooling Layer	—	$8 \times 8 \times 128$
Convolution Layer	$3 \times 3 \times 256, s = 1, p = 1$	$8 \times 8 \times 256$
Activation Layer	—	$8 \times 8 \times 256$
Pooling Layer	—	$4 \times 4 \times 256$
Fully-Connected Layer	4096×256	1×256
Fully-Connected Layer	256×10	1×10

it. For the 128-bit security level, we set $key_length = 3072$ bits. To significantly enhance the computational efficiency of the Paillier algorithm, we leverage the Chinese Remainder Theorem for optimizing the client-side decryption process, resulting in a notable speedup of approximately 1.5 times.

Two widely recognized datasets are employed for our experiments: the MNIST dataset [32] and the CIFAR-10 dataset [33]. The MNIST dataset contains 70,000 grayscale images of handwritten digits. Each grayscale image has dimensions of $28 \times 28 \times 1$ pixels and is labeled with one of the 10 digits from 0 to 9. On the other hand, the CIFAR-10 dataset consists of 60,000 RGB images. These images have dimensions of $32 \times 32 \times 3$ pixels, and each is labeled with one of the 10 category labels.

The model architectures employed for the experiments are outlined in Tables IV and V. For the above two datasets, we conduct tests using two distinct network architectures, denoted Network I and Network II. The experimental process is as follows: First, model parameters are iteratively optimized using training data to create two inference models with high accuracy. The resulting models are hosted by the server $\mathcal{S}$. Following the steps outlined in Section III-A, the client $\mathcal{C}$ encrypts the test data for querying. Subsequently, the server $\mathcal{S}$ performs the ciphertext CNN inference with the help of the client $\mathcal{C}$, involving the CNN layers as described in the algorithms from Sections III-B, III-C, III-D, and III-F.

TABLE VI
ACCURACY COMPARISON OF DIFFERENT NETWORKS ON MNIST AND CIFAR-10

Scheme	MNIST	CIFAR-10
Plaintext	98.15%	84.78%
Plaintext (Square approximation)	95.22%	79.54%
PCCS (Sigmoid) [23]	88.32%	37.25%
PCCS (Softplus) [23]	94.64%	65.37%
PCCS (Tanhplus) [23]	96.31%	78.69%
EPCNN	97.87%	83.46%

B. Accuracy Evaluation

In Table VI, we apply the EPCNN framework to Network I on MNIST and Network II on CIFAR-10 for inference accuracy tests. The results are compared against the plaintext model and various polynomial approximation methods.

The plaintext inference scheme achieves high accuracy rates of 98.15% on MNIST and 84.78% on CIFAR-10. In contrast, while simple square-function approximations can enhance computational efficiency, they significantly compromise accuracy. PCCS selects three approximation functions—Sigmoid, Softplus, and their proposed Tanhplus—to replace ReLU and further expanded them into fourth-order Taylor series. While the Sigmoid function performs adequately on MNIST, it exhibits poor performance when applied to the CIFAR-10 dataset, indicating it is unsuitable for complex CNN architectures. By comparison, their proposed Tanhplus function, which is continuously differentiable, outperforms the Softplus function in classification accuracy. However, higher-order expansions, while improving accuracy, incur substantial computational overhead.

EPCNN achieves accuracy rates of 97.87% and 83.46%, showing only a slight reduction compared to plaintext inference but better than all polynomial approximation methods. This minor accuracy loss can be attributed to the fact that the Paillier homomorphic encryption can only operate on integer data, while the model parameters $\mathcal{C}$ and $\mathcal{W}$ are matrices composed of floating point numbers. Therefore, some precision loss is inevitable when converting floating point numbers into integers. Theoretically, opting for a larger rounding factor k could mitigate this accuracy loss and bring EPCNN's inference accuracy closer to plaintext inference. However, increasing k also amplifies the computational cost of homomorphic scalar multiplications on rounded model parameters. Balancing these factors, we chose a rounding factor of $k = 10^4$ in EPCNN.

Notably, all schemes exhibit higher accuracy on MNIST compared to CIFAR-10. This discrepancy arises because Network II possesses a deeper and more complex structure compared to Network I, thereby amplifying error as they propagate through

TABLE VII
PERFORMANCE OF EPCNN AT EACH LAYER OF NETWORK I

Layer	Runtime (s)	Communication (KB)
Initialization	0.855	8.531
Convolution Layer	7.236	—
Activation Layer	2.598	273
Pooling Layer	0.001	—
Convolution Layer	57.895	—
Activation Layer	1.291	161
Pooling Layer	0.001	—
Fully-Connected Layer	0.465	—
Final Inference Decryption	0.003	0.180
Total	70.345	442.711

TABLE VIII
PERFORMANCE OF EPCNN AT EACH LAYER OF NETWORK II

Layer	Runtime (s)	Communication (KB)
Initialization	1.721	29.25
Convolution Layer	27.525	—
Activation Layer	7.107	1248
Pooling Layer	0.007	—
Convolution Layer	293.601	—
Activation Layer	3.542	736
Pooling Layer	0.002	—
Convolution Layer	290.859	—
Activation Layer	1.807	480
Pooling Layer	0.001	—
Fully-Connected Layer	20.438	—
Fully-Connected Layer	0.039	—
Final Inference Decryption	0.003	0.180
Total	646.652	2493.43

layers. EPCNN effectively mitigates this precision loss. A key advantage of EPCNN is that it avoids the approximate operation of RELU activation function, consequently preventing further amplification of accuracy loss. This is the primary reason why EPCNN consistently outperforms the polynomial approximation model.

In summary, the findings highlight the effectiveness of EPCNN in maintaining inference accuracy while leveraging homomorphic encryption to protect sensitive information.

C. Efficiency Evaluation

In this part, we assess the runtime and communication overhead associated with EPCNN on both Network I and Network II, as presented in Tables VII and VIII. On Network I, EPCNN completes ciphertext inference on MNIST images in approximately 70.345 seconds and 442.711 Kbytes. However, the task of executing ciphertext inference on CIFAR-10 images on Network II requires substantially more time, amounting to 646.652 seconds, and a significantly larger data volume of 2493.43 Kbytes. From these tables, it's evident that the runtime of EPCNN is mainly concentrated in the convolutional layers. This phenomenon arises due to a large number of homomorphic scalar multiplications that the server $\mathcal{S}$ needs to perform in this kind of layer to realize the convolution operation. These homomorphic scalar multiplications encompass modular exponentiation operations on ciphertexts, which is extremely time-consuming. For instance, examining the second convolutional layer in both Tables VII and VIII, the server $\mathcal{S}$ undertakes 100352 and

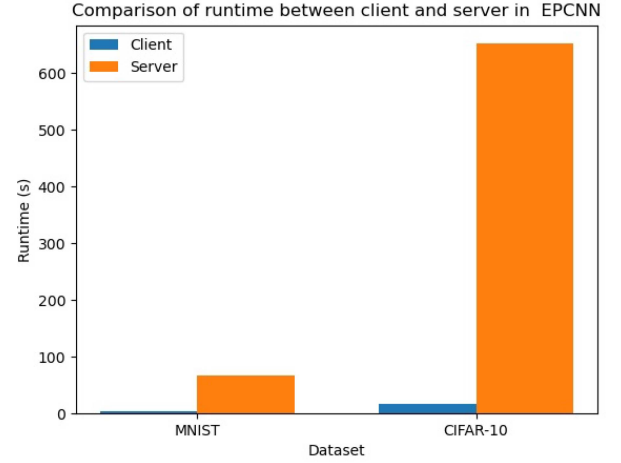


Fig. 2. Comparison of runtime between client and server in EPCNN.

2097152 ciphertext convolution operations, respectively. In contrast, the homomorphic additions executed in the pooling layer demand only a trivial amount of time. Conversely, communication overhead primarily comes from the activation layer. The magnitude of this overhead is contingent on the quantity and dimensions of the blinded convolution matrices and label matrices necessitated for exchange between the server $\mathcal{S}$ and client $\mathcal{C}$. In summation, the convolutional layer dominates EPCNN's runtime due to the computational intensity of homomorphic scalar multiplications. Communication overhead chiefly comes from the activation layer, contingent on the dimensions and quantities of matrices transferred between $\mathcal{S}$ and $\mathcal{C}$.

To assess the practical feasibility of outsourcing, we undertake a comparison between the execution times of client $\mathcal{C}$ and server $\mathcal{S}$ in EPCNN, as illustrated in Fig. 2. On the MNIST dataset, the client $\mathcal{C}$ completes its execution in approximately 3.725 seconds, while on the CIFAR-10 dataset, it takes around 16.576 seconds. Meanwhile, server $\mathcal{S}$ requires 67.223 seconds and 651.481 seconds to perform the respective executions on the same datasets. The execution time of client $\mathcal{C}$ is mainly influenced by encryption and decryption operations, with the time spent on generating the label matrices being virtually negligible in comparison. Conversely, the time overhead incurred by server $\mathcal{S}$ mainly arises from homomorphic scalar multiplications. The maximum number of modular exponentiation operations executed by $\mathcal{S}$ can reach to several tens of millions, a computational load that would be infeasible for client $\mathcal{C}$ to manage. Through the security outsourcing, the server $\mathcal{S}$ achieves a substantial reduction in computing time for client $\mathcal{C}$, amounting to a speedup factor of approximately $18\times$ to $39\times$. This improvement aligns with the design objective of making outsourcing practically viable.

Subsequently, we conduct an in-depth investigation into how various factors affect the performance of EPCNN, encompassing the Paillier homomorphic encryption algorithm's modulus N , the number n and the dimension d of query images, as visualized in Figs. 3 and 4. From the figures, it's evident that the runtime of EPCNN is linearly related to the number n and positively correlates with the modulus N . Moreover, the runtime exhibits

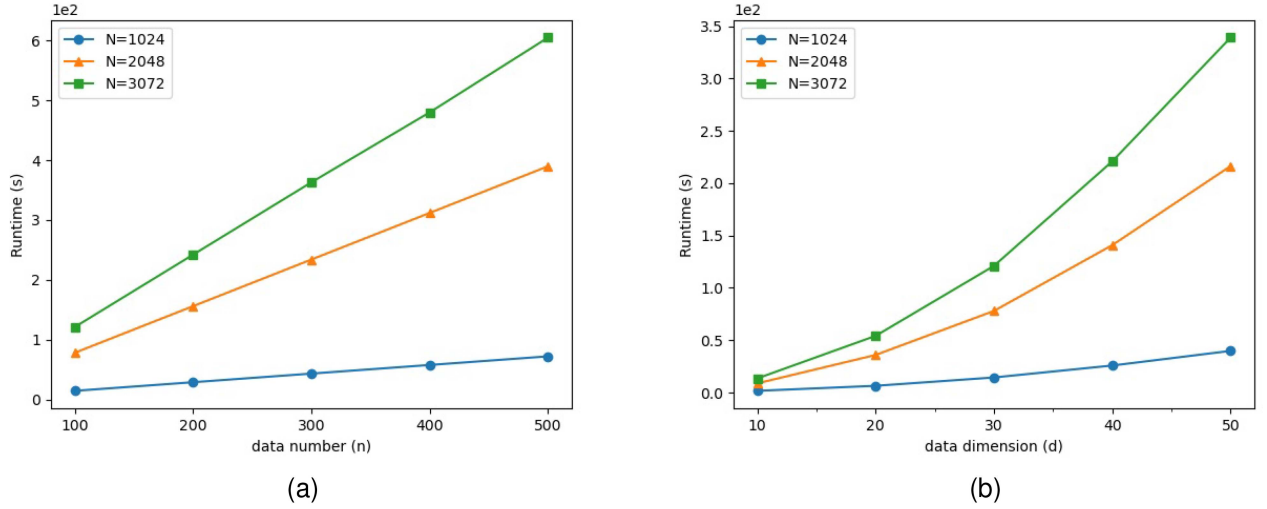


Fig. 3. Effect of modulus N , number of data n and data dimension d on running time. (a) When $d = 30$, the effect of the modulus N on the running time under different data volumes. (b) When $n = 100$, the effect of the modulus N on the running time under different data dimensions.

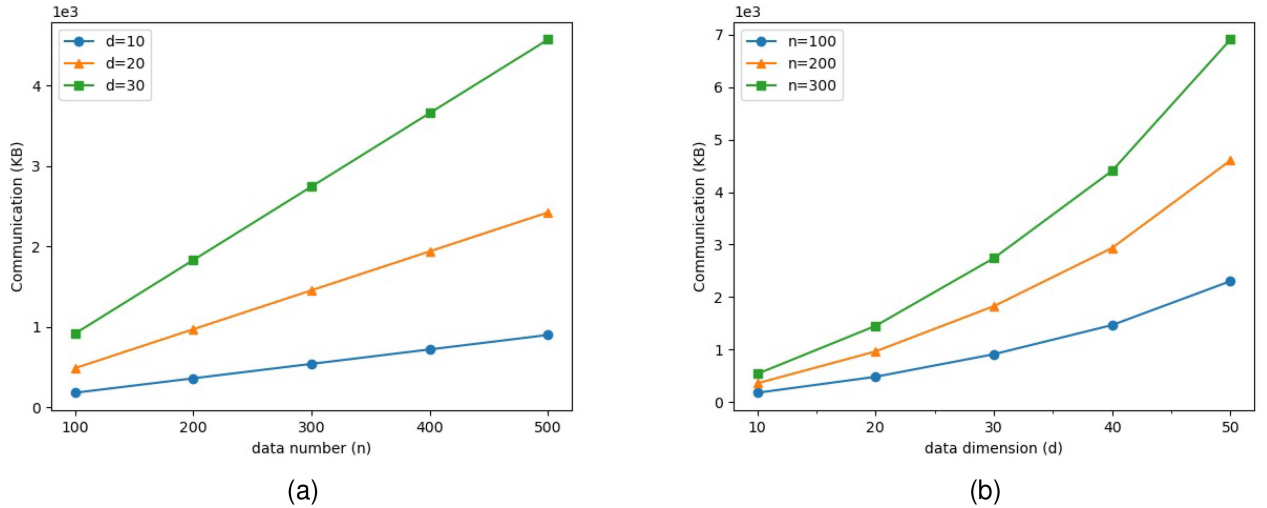


Fig. 4. Effect of number of data n and data dimension d on communication overhead. (a) The relationship between the number of data n and the communication overhead. (b) The relationship between data dimension d and the communication overhead.

an exponential rise concerning the dimension d . In contrast, the communication cost of EPCNN is contingent solely on the query image and lacks an apparent connection with the modulus N . As depicted in the figures, the communication cost exhibits a linear relationship with the number n and demonstrates exponential growth with the dimension d .

In the concluding phase of our evaluation, we conduct a comparative assessment of EPCNN against several representative schemes. Given the variability in network structures across these schemes, for the sake of uniformity, we all adopt the same network architectures established in the existing literature [6], referred to as Network III in Table IX and Network IV in Table X. Table XI meticulously provides the comparative performance evaluation of various privacy-preserving CNN inference schemes concerning their runtime and communication overhead.

In Network III, EPCNN completes the entire inference process in only 0.54 seconds, marking a significant efficiency

TABLE IX
NETWORK III: CNN ARCHITECTURE FOR COMPARISON III

Layer	Parameters	Output size
Convolution Layer	$5 \times 5 \times 5, s = 2, p = 1$	$13 \times 13 \times 5$
Activation Layer	—	$13 \times 13 \times 5$
Pooling Layer	—	1×100
Activation Layer	—	1×100
Fully-Connected Layer	100×10	1×10

improvement compared to CryptoNets and LPFHE, both of which also rely solely on homomorphic encryption to perform ciphertext CNN inference. Gazelle and Pio, as lightweight secret-sharing schemes, achieve the lowest computational cost, with runtimes of 0.03 seconds and 0.01 seconds, respectively. However, this efficiency is achieved through specific trade-offs: Gazelle compromises on model parameter protection, while Pio relies on a dual-server architecture, which poses challenges

TABLE X
NETWORK IV: CNN ARCHITECTURE FOR COMPARISON IV

Layer	Parameters	Output size
Convolution Layer	$5 \times 5 \times 16, s = 1, p = 0$	$24 \times 24 \times 16$
Activation Layer	—	$24 \times 24 \times 16$
Pooling Layer	—	$12 \times 12 \times 16$
Convolution Layer	$5 \times 5 \times 16, s = 1, p = 0$	$8 \times 8 \times 16$
Activation Layer	—	$8 \times 8 \times 16$
Pooling Layer	—	$4 \times 4 \times 16$
Fully-Connected Layer	256×100	1×100
Activation Layer	—	1×100
Fully-Connected Layer	100×10	1×10

TABLE XI
COMPARISON OF RUNNING TIME AND COMMUNICATION OVERHEAD OF
DIFFERENT SCHEMES

Network	Scheme	Runtime (s)	Communication (MB)
III	CryptoNets [12]	297.50	372.20
	Gazelle [17]	0.03	0.50
	Pio [21]	0.01	0.40
	LPFHE [19]	9.53	9.25
	EPCNN	0.54	0.03
IV	MUSE [18]	23.47	4280.32
	Sonic [20]	13.60	10.80
	Zhao et al. [22]	3.15	0.372
	PCCS [23]	236.83	3266.24
	EPCNN	16.67	0.220

for practical deployment. In terms of communication overhead, EPCNN demonstrates a significant reduction compared to similar homomorphic encryption schemes, requiring only 0.03 MB, a stark contrast to the 372.20 MB of CryptoNets and the 9.25 MB of LPFHE. Notably, EPCNN also achieves an advantage over secret-sharing schemes such as Gazelle and Pio by minimizing the number of client-server interactions, highlighting its design focus on reducing communication costs while maintaining strong privacy guarantees.

For Network IV, EPCNN achieves a runtime of 16.67 seconds, comparable to Sonic's 13.60 seconds and MUSE's 23.47 seconds, while significantly outperforming PCCS, which requires 236.83 seconds due to the additional overhead of decrypting ciphertext twice in its DT-PKC cryptosystem. However, EPCNN's runtime is less efficient than Zhao et al.'s scheme, which completes the task in 3.15 seconds by leveraging secret sharing between two servers. This runtime improvement comes at the expense of security, as the scheme by Zhao et al. does not provide full privacy guarantees. Regarding communication overhead, EPCNN achieves substantial reductions compared to MUSE and PCCS, requiring only 0.220 MB. By avoiding additional zero-knowledge proofs and frequent interactions, EPCNN demonstrates a significant advantage in communication efficiency.

In summary, EPCNN emphasizes security and practicality while maintaining acceptable performance trade-offs, effectively achieving a balance among various privacy-preserving CNN inference schemes.

VI. CONCLUSION

In this paper, we introduce EPCNN, an efficient and practical privacy-preserving CNN inference. Our approach ensures the

confidentiality of client query data, inference outcomes, and the server's model parameters while minimizing computational and communication burdens for clients. Moreover, we tackle the challenge of non-linear operations on ciphertexts by allowing the client to judge the sign of blinded convolution results. This approach improves efficiency without compromising accuracy. Our security analysis proves the strong privacy-preserving capability of EPCNN. In comparison to existing methods, our scheme stands out as more practical and superior in performance.

REFERENCES

- [1] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, and F. Alsaadi, "A survey of deep neural network architectures and their applications," *Neurocomputing*, vol. 234, pp. 11–26, 2017.
- [2] M. Ribeiro, K. Grolinger, and M. Capretz, "MLaaS: Machine learning as a service," in *Proc. IEEE 14th Int. Conf. Mach. Learn. Appl.*, 2015, pp. 896–902.
- [3] Z. A. E. Houda, A. Hafid, L. Khoulkhi, and B. Brik, "When collaborative federated learning meets blockchain to preserve privacy in healthcare," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2455–2465, Sep./Oct. 2023.
- [4] B. Rouhani, M. Riazzi, and F. Koushanfar, "DeepSecure: Scalable provably-secure deep learning," in *Proc. 55th Annu. Des. Automat. Conf.*, 2018, pp. 1–6.
- [5] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. 2017 IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
- [6] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [7] D. Rathee et al., "Cryptflow2: Practical 2-party secure inference," in *Proc. 2020 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [8] Z. Huang, W. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 809–826.
- [9] L. K. Ng and S. S. Chow, "GForce: GPU-friendly oblivious and rapid neural network inference," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2147–2164.
- [10] L. Ng and S. Chow, "SoK: Cryptographic neural-network computation," in *Proc. 2023 IEEE Symp. Secur. Privacy*, 2023, pp. 497–514.
- [11] D. Kim and C. Guyot, "Optimized privacy-preserving CNN inference with fully homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2175–2187, 2023.
- [12] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.
- [13] M. Riazzi, C. Weinert, O. Tkachenko, E. Songhori, T. Schneider, and F. Koushanfar, "Chameleon: A hybrid secure computation framework for machine learning applications," in *Proc. 2018 Asia Conf. Comput. Commun. Secur.*, 2018, pp. 707–721.
- [14] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 35–52.
- [15] M. Riazzi, M. Samragh, H. Chen, K. Laine, K. Lauter, and F. Koushanfar, "Xonn: Xnor-based oblivious deep neural network inference," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1501–1518.
- [16] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Proc. 11th Annu. Int. Cryptol. Conf. Adv. Cryptol.*, 1992, pp. 420–432.
- [17] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "Gazelle: A low latency framework for secure neural network inference," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1651–1669.
- [18] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. Popa, "Muse: Secure inference resilient to malicious clients," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2201–2218.
- [19] J. Wan, D. Li, J. Fang, and Z. Jiang, "LPFHE: Low-complexity polynomial CNNs for secure inference over FHE," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2024, pp. 403–423.
- [20] X. Liu, Y. Zheng, X. Yuan, and X. Yi, "Securely outsourcing neural network inference to the cloud with lightweight techniques," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 620–636, Jan./Feb. 2022.

- [21] X. Yang, J. Chen, K. He, H. Bai, C. Wu, and R. Du, "Efficient privacy-preserving inference outsourcing for convolutional neural networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4815–4829, 2023.
- [22] R. Zhao, Y. Xie, D. He, K. Choo, and Z. Jiang, "Practical privacy-preserving convolutional neural network inference framework with edge computing for health monitoring," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5995–6006, Nov./Dec. 2024.
- [23] B. Wang et al., "Privacy-preserving convolutional neural network classification scheme with multiple keys," *IEEE Trans. Serv. Comput.*, vol. 17, no. 01, pp. 322–335, Jan./Feb. 2024.
- [24] N. Smart and F. Vercauteren, "Fully homomorphic SIMD operations," *Des. Codes Cryptogr.*, vol. 71, pp. 57–81, 2014.
- [25] W. Srinivasan, P. Akshayaram, and P. R. Ada, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th USENIX Secur. Symp.*, 2019, pp. 2505–2522.
- [26] K. Huang, X. Liu, S. Fu, D. Guo, and M. Xu, "A lightweight privacy-preserving CNN feature extraction framework for mobile sensing," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1441–1455, May/Jun. 2021.
- [27] M. Li, S. Chow, S. Hu, Y. Yan, C. Shen, and Q. Wang, "Optimizing privacy-preserving outsourced convolutional neural network predictions," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1592–1604, May/Jun. 2022.
- [28] X. Liu, B. Wu, X. Yuan, and X. Yi, "Leia: A lightweight cryptographic neural network inference system at the edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 237–252, 2022.
- [29] J. Wang, D. He, A. Castiglione, B. Gupta, M. Karupiah, and L. Wu, "PC-NNCEC: Efficient and privacy-preserving convolutional neural network inference based on cloud-edge-client collaboration," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2906–2923, Sep./Oct. 2023.
- [30] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptogr. Techn.*, 1999, pp. 223–238.
- [31] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, 2021.
- [32] G. Cohen, S. Afshar, J. Tapson, and A. V. Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. 2017 Int. Joint Conf. Neural Netw.*, 2017, pp. 2921–2926.
- [33] H. Li, H. Liu, X. Ji, G. Li, and L. Shi, "CIFAR10-DVS: An event-stream dataset for object classification," *Front. Neurosci.*, vol. 11, pp. 309–309, 2017.



Gang He (Graduate Student Member, IEEE) received the B.S. degree in communication engineering in 2020 from Shanghai University, Shanghai, China, where he is currently working toward the Ph.D. degree with the School of Communications and Information Engineering. His research interests include cryptography and privacy-preserving machine learning.



Yanli Ren received the M.S. degree in applied mathematics from Shaanxi Normal University, Xi'an, China, in 2005, and the Ph.D. degree in computer science and technology from Shanghai Jiao Tong University, Shanghai, China, in 2009. She is currently a Professor with the School of Communication and Information Engineering, Shanghai University, Shanghai, China. She has authored or coauthored more than 80 quality papers, including publications in IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON CLOUD COMPUTING and AsiaCCS. Her research interests include applied cryptography, secure outsourcing computation, blockchain security, AI security, and network security.



Jun Zhao (Member, IEEE) received the bachelor's degree from Shanghai Jiao Tong University, Shanghai, China, in July 2010, and the Ph.D. degree in electrical and computer engineering from Carnegie Mellon University (CMU), Pittsburgh, PA, USA. In May 2015, he affiliated with CyLab Security & Privacy Institute, CMU. He is currently an Assistant Professor with the College of Computing and Data Science, formerly known as the School of Computer Science and Engineering, Nanyang Technological University (NTU), Singapore. Before joining NTU first as a Postdoctoral Researcher and then as a Faculty Member, he was a Postdoctoral Researcher with Arizona State University, Tempe, AZ, USA, as an Arizona Computing Postdoctoral Best Practices Fellow. He is also the Editor of IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and other journals. He was selected as the Best Editor of IEEE Wireless Communications Letters and the Best Editor of IEEE Transactions on Cognitive Communications and Networking in 2023.



Guorui Feng received the B.S. and M.S. degree in computational mathematic from Jilin University, Changchun, China, in 1998 and 2001, respectively, the Ph.D. degree in electronic engineering from Shanghai Jiaotong University, Shanghai, China, 2005. From January 2006 to December 2006, he was an Assistant Professor with East China Normal University, Shanghai. In 2007, he was a Research Fellow with Nanyang Technological University, Singapore. He is currently with the School of Communication and Information Engineering, Shanghai University, Shanghai. His research interests include image processing, image analysis, and computational intelligence.



Xinpeng Zhang received B.S. degree from Jilin University, Changchun, China, in 1995, and the M.S. and Ph.D. degrees from Shanghai University, Shanghai, China, in 2001 and 2004, respectively. Since 2004, he has been with the Faculty of the School of Communication and Information Engineering, Shanghai University, where he is currently a Fulltime Professor. He is also with the Faculty of the School of Computer Science, Fudan University, Shanghai. From 2010 to 2011, he was with The State University of New York at Binghamton, Binghamton, NY, USA, as a Visiting Scholar and also with Konstanz University, Konstanz, Germany, as an Experienced Researcher, sponsored by the Alexander von Humboldt Foundation from 2011 to 2012. His research interests include multimedia security, image processing, and digital forensics. He has authored or coauthored more than 200 research papers. From 2014 to 2017, he was an Associate Editor for IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY.